

# Your first C++ savings calculation

Daily learning note | 23 September 2026 | C001

Sidiq review status: Not yet reviewed.

## Research question and hypothesis

Can a small compiled C++ program reproduce a one-year savings calculation using named values and clear output? For an invented opening balance of 1,000 units and a 5% effective annual rate, our hypothesis is 50 units of interest and a closing balance of 1,050 units. This is a synthetic arithmetic exercise.

The Wednesday C++ track begins here, following the requested gradual finance learning plan. Yesterday's Q002 work addressed label leakage in Python; today's connection is the same discipline of stating a contract and checking it against an independent example. C++ is being learned, not used to claim a speedup. Q003 experiment manifests remains the next non-C++ task.

## Beginner explanation and financial terms

Imagine an account containing 1,000 tokens. At the end of one year, add five tokens for every hundred original tokens. There are ten groups of one hundred, so the addition is 50 tokens. Nothing else happens. The computer performs that arithmetic; we provide the assumptions and verify the answer.

**Principal P:** opening amount, measured in currency units. **Rate r:** interest per unit of principal for the stated period. **Interest I:** the extra amount earned during the period. **Closing balance B:** opening principal plus interest. **Effective annual rate:** proportional growth over one full year under the model. **Synthetic fixture:** invented inputs and expected results used to check software, not observed market data.

## Mathematics and worked example

Percent means per hundred:  $r = 5 / 100 = 0.05$ . The computation is  $I = P * r$  and  $B = P + I = P * (1 + r)$ . The rate is a dimensionless fraction; multiplying it by a balance in units produces interest in units. The one-year period is implicit in  $r$ . No extra multiplication by 365 is needed.

| Quantity        | Hand calculation      | Expected output |
|-----------------|-----------------------|-----------------|
| Opening balance | Given synthetic input | 1000.00 units   |
| Annual rate     | $5 / 100 = 0.05$      | 5.00 percent    |
| Interest        | $1000 * 0.05$         | 50.00 units     |
| Closing balance | $1000 + 50$           | 1050.00 units   |

An independent mental check is  $(1000 / 100) * 5 = 10 * 5 = 50$ . Entering 5.0 as the fractional rate would mean 500%, producing 5,000 units of interest and a 6,000-unit balance. Units matter before syntax.

## How the C++ program works

The source is `cpp/projects/savings_growth/main.cpp`. Read its statements from top to bottom. The first two include directives make standard input/output and formatting declarations available. `int main()` names the program's entry function. Its braces contain the statements to execute.

`const double principal = 1000.0;` names a floating-point value. `const` prevents reassignment, `double` selects the number type, `=` initializes it, and `;` ends the statement. The next statement names the fractional rate 0.05. Two more statements multiply `principal` by rate, then add interest to `principal`. Keeping interest separate makes the intermediate answer visible for checking.

`std::cout` is the standard output stream. The `<<` operator inserts text or values into that stream. The `std::` prefix identifies names in the standard library. Quoted text is printed literally; `'\n'` inserts a newline. `return 0;` tells the operating system that the program completed successfully.

`std::fixed` and `std::setprecision(2)` together request two digits after the decimal point. The rate is multiplied by 100 only when displayed as a percentage; the calculation uses 0.05. Formatting changes how numbers are printed. It does not turn double into exact decimal arithmetic.

## From source file to executable

A compiler translates the source and links the needed library support into a runnable program. Our build explicitly selects C++17. The warning flags are `-Wall`, `-Wextra` and `-Wpedantic`; `-Werror` makes warnings fail the build. A clean compilation is useful evidence, but it does not prove the financial formula.

From the repository root, run `make cpp-savings` to build and print the example. Run `make cpp-check` to compare it with the hand-authored expected output, or `make check` for both Python and C++ checks. Run `make note NOTE_DATE=2026-09-23` to regenerate this teaching PDF.

The executable is `build/cpp/savings_growth/savings`. Build products and captured output are ignored by Git. The project README also gives a direct compiler command. A compiler can be selected explicitly with `make -B cpp-check CXX=c lang++; -B` rebuilds even if a binary already exists.

## Input contract and evaluation protocol

There is exactly one fixture: principal 1000.0, rate 0.05, and one full model year, without calendar dates. Both fixed inputs are finite and nonnegative. There are no deposits, withdrawals, fees, taxes or inflation adjustments. No rate was fetched from a bank, and no market-data period is represented.

The program has no external input interface. Runtime validation of supplied amounts is therefore not applicable yet; C002 and C003 introduce validated function parameters and command-line parsing respectively. Passing command-line arguments does not change this example's constants.

The predeclared metric is agreement with every line in `expected.txt`, including units and two-decimal amounts. Compiler failure, unsuccessful process exit or any output difference fails `cpp-check`. The expected absolute difference for each displayed amount is zero. No model is fitted or tuned.

## Observed results and validation

| Check                        | Observed result                                           |
|------------------------------|-----------------------------------------------------------|
| Compiled example             | 50.00 interest; 1050.00 closing balance                   |
| Displayed amount differences | Zero against the hand calculation                         |
| Repeatability                | Two runs matched expected.txt exactly                     |
| Wrong-rate diagnostic        | A temporary rate of 5.0 disagreed, as expected            |
| Existing Python suite        | 49 tests passed                                           |
| C++ acceptance check         | Golden-output comparison passed                           |
| Lint, formatting, types      | Ruff passed; 19 files formatted; mypy passed for 13 files |
| Dependency consistency       | No broken requirements found                              |

The compiler was Apple Clang 21.0.0 targeting arm64 macOS. No new library dependency was added. The C++ check now belongs to make check, which the existing continuous-integration workflow runs. Local success is not a claim about a remote CI run; publication status is reported separately. There is no measured code-coverage percentage or benchmark speedup.

The wrong-rate diagnostic compiled a separate temporary copy, leaving the real source unchanged. It printed 5,000.00 interest and disagreed with the expected fixture. This demonstrates that the check can detect a realistic percentage-unit error. It does not validate every possible implementation error.

## Limitations and interpretation

This fixture establishes one displayed calculation, not a general savings calculator. Other amounts, negative rates, zero horizons, malformed input, overflow and rounding boundaries need later tests when those cases are supported. Two decimals in output can hide smaller numerical errors.

Many decimal fractions, including 0.05 on this implementation, are not represented exactly in binary floating-point. The correct displayed answer here is compatible with that limitation. Financial accounting software requires an explicit representation and rounding policy; this example does not supply one.

Simple interest and annual compounding give the same answer for one year. For multiple years, simple interest uses  $P * (1 + n * r)$ , whereas annual compounding uses  $P * (1 + r)^n$ . At two years and 5%, these would be 1,100 and 1,102.50 respectively. This is an illustrative derivation, not a feature implemented today.

No observed saving product, investment recommendation, realised return, or predictive skill follows from these invented inputs. There is no comparison with Python performance and no production acceleration claim.

## What to explain in an interview

State the model first: one year, a fractional effective annual rate, and no cash flows or costs. Derive interest and closing wealth with their units. Then explain the source-to-executable step, const double, and output formatting. Show the hand-calculated acceptance test and the wrong-rate diagnostic. Finish with its scope: a verified first example, with multi-period inputs and rounding still to be designed.

## Primary sources and implementation decisions

Official documentation consulted on 23 September 2026. The arithmetic fixture and its expected values were designed here; these sources support C++ language and tool behaviour.

1. Microsoft, Built-in types (C++). Explains floating-point types and finite precision. This informs our choice of double for a beginner exercise and the explicit limitation on exact decimal money.  
<https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp?view=msvc-170>
2. Microsoft, ios functions and iomanip functions. Documents fixed notation and setprecision. We combine them to print readable two-decimal amounts without claiming that formatting changes stored values. <https://learn.microsoft.com/en-us/cpp/standard-library/ios-functions?view=msvc-170>  
<https://learn.microsoft.com/en-us/cpp/standard-library/iomanip-functions?view=msvc-170>
3. Clang Compiler User's Manual. Documents explicit language modes and compiler diagnostics, including warnings treated as errors. Our compile command sets the language standard and enables common warnings. <https://clang.llvm.org/docs/UsersManual.html>

## Next eligible work

Thursday resumes Q003: versioned experiment manifests and checksums with required question, data kind, source hash and evaluation protocol. C002 is ready for the next Wednesday slot, 30 September: move savings growth into a function, state annual compounding, and test zero rate, zero years and invalid inputs. Q005a's existing availability timestamp-fold defect remains open; use UTC-normalized inputs meanwhile. Progress continues independently of personal review, which remains Not yet reviewed.

## Check your understanding

1. Why does the program use 0.05 in the calculation and 5.00 in the displayed percentage?
2. What do const, double and the semicolon mean in the principal declaration?
3. Does printing two decimal places make the stored value exact to a cent?
4. Why can this one-year result not choose between simple interest and annual compounding?
5. What does the golden-output test establish, and what does the wrong-rate diagnostic add?

## Suggested answers

1. A percentage is per hundred, so 5% is the fractional multiplier 0.05. Multiplying by 100 converts the fraction back to the displayed percentage. Using 5.0 in the calculation means 500%.
2. const prevents reassignment, double is the floating-point type, and the semicolon ends the statement. The name identifies the value so later statements can use it without repeating the number.
3. No. Formatting controls output. Binary floating-point can approximate decimal values, so an exact-money system needs a separate representation and rounding contract.
4. At  $n = 1$ , both  $P * (1 + n * r)$  and  $P * (1 + r)^n$  equal  $P * (1 + r)$ . A longer horizon and an explicit compounding convention are needed to distinguish them.
5. It establishes that this executable's displayed result matches this fixed hand calculation. The diagnostic shows it rejects one realistic unit error; neither check proves correctness for arbitrary inputs.